

UML-діаграми проєкту

Remote File Folder Client

(спрощений аналог Google Drive)

Уточкіна Яна

Етап: Проєктування архітектури та поведінки системи
15 вересня 2026 р.

Анотація

У документі представлено комплект UML-діаграм для проєкту **Remote File Folder Client** — застосунку для роботи з віддаленою файловою текою (спрощений аналог Google Drive) з веб- та десктоп-клієнтами, Гобекендом, PostgreSQL та синхронізацією локальної теки, побудованою на серверному версійному курсорі й локальному скануванні на клієнті. Діаграми описують систему такою, якою вона реально реалізована, і впорядковані відповідно до життєвого циклу проєктування: від аналізу вимог і високорівневої архітектури до деталізованого проєктування окремих підсистем. Усі діаграми побудовано в нотації Mermaid.

Зміст

1	Аналіз вимог	2
2	Архітектурне проєктування (високий рівень)	3
2.1	Діаграма прецедентів	3
2.2	Діаграма класів (загальна структура)	4
2.3	Діаграма розгортання (загальна архітектура)	7
3	Деталізоване проєктування	8
3.1	ВОРС-діаграма класів для прецедента “Синхронізувати локальну теку з віддаленою”	9
3.2	Діаграма об’єктів (знімок стану синхронізації)	11
3.3	Діаграми взаємодії (Sequence & Communication Diagrams)	13
3.4	Діаграма станів	21

3.5	Діаграма компонентів	23
3.6	Діаграма пакетів	24
3.7	Діаграма активності	25
4	Висновки	26

1. Аналіз вимог

Система надає користувачу автентифікований доступ до персональної віддаленої теки (“віртуального диска”), у якій він може переглядати список файлів разом з їх атрибутами (ім’я, дата створення, дата модифікації, автор вивантаження, автор останньої редакції), вивантажувати, завантажувати та видаляти файли, а також синхронізувати обраний локальний каталог із віддаленим.

Ключові функціональні вимоги, як їх реально реалізовано:

- реєстрація та автентифікація користувача на віддаленому сервері (username/пароль, JWT access + refresh токени);
- перегляд атрибутів файлів із можливістю показу/приховування стовпців (стовпець імені файлу приховати не можна);
- перегляд вмісту файлів окремих типів: .java – як текст, .png – як зображення;
- сортування файлів за будь-яким стовпцем – ім’я, дата створення, дата модифікації, автор вивантаження або останній редактор (за зростанням / спаданням);
- фільтрація файлів за розширенням через випадний список, що формується з розширень, які реально присутні серед файлів користувача;
- вивантаження або одним запитом multipart/form-data, або, для великих файлів, chunked-сесією (initiate → $N \times \text{chunk}$ → complete), яка утримує кожен окремий HTTP-запит у межах ліміту розміру тіла безкоштовного тарифу Cloudflare Tunnel;
- синхронізація локальної теки з віддаленою: сервер повідомляє, що змінилося від останньої відомої клієнту версії (лише diff download/delete_local) а клієнт самостійно узгоджує власні локальні зміни (нові/змінені/видалені файли), вивантажуючи, повторно вивантажуючи або видаляючи їх на сервері.

Примітка щодо обсягу. У REQUIREMENTS.md спочатку був накреслений повніший протокол, у якому клієнт також надсилає локальний маніфест (name, size, mtime, sha256 для кожного файлу), щоб сервер міг виявляти справжні двосторонні конфлікти редагування та пропонувати розв’язання “keep local / keep remote / keep both”. Реалізований ендпоінт POST /api/sync/diff приймає це поле маніфесту для сумісності наперед, але не читає й не порівнює його – це свідоме, задокументоване звуження обсягу (див. backend/internal/service/sync.go та desktop/.../SyncViewModel.swift) а не недогляд. Відповідно, якщо той самий файл змінюється і в локальній теці, і на сервері між двома циклами синхронізації, виявлення конфлікту чи запиту до користувача не відбувається: перемагає та половина процедури узгодження на клієнті, яка виконується першою (last-writer-wins). Діаграми цього звіту описують систему саме такою, якою її реалізовано, включно з цим обмеженням, а не первісно накреслений дизайн розв’язання конфліктів.

Архітектурно система складається з React-клієнта (веб, який роздає статичний контейнер nginx), Swift-клієнта (macOS, зі спостерігачем локаль-

ної теки на основі FSEvents та періодичним опитуванням), Go API-сервера (шари handler / service / repository), PostgreSQL для метаданих та абстракції FileStorage, наразі реалізованої лише локальним диском (S3/R2-сумісна реалізація – задокументована точка розширення на майбутнє, яку ще не написано), розгорнутих на домашньому Ubuntu-сервері та опублікованих через Cloudflare Tunnel.

На основі цих вимог у наступних розділах побудовано діаграму прецедентів, а також архітектурні та деталізовані діаграми, що описують структуру і поведінку системи.

2. Архітектурне проектування (високий рівень)

На цьому етапі визначається загальна архітектура системи: хто є її користувачами і які сценарії вона підтримує (діаграма прецедентів), яка загальна структура даних (діаграма класів високого рівня) та яка фізична топологія розгортання (діаграма розгортання).

2.1 Діаграма прецедентів

Діаграма (рис. 1) показує єдиного актора *Користувача*, який працює із системою через веб- або десктоп-клієнт, та основні прецеденти, що відповідають функціональним вимогам розділу 1, включно з “Зареєструватися” як окремою точкою входу. Частина операцій над файлами включають («include») прецедент “Автентифікуватися”, оскільки доступ до робочої області можливий лише після успішного входу. Перегляд вмісту файлу *розширюється* («extend») двома альтернативними сценаріями відображення (.java / .png). На відміну від попередньої версії цієї діаграми, прецедент “Розв’язати конфлікт синхронізації”, що розширював би синхронізацію, відсутній – як зазначено в примітці розділу 1, у реалізованій системі конфлікт ніколи не виявляється й не показується користувачу.

Примітка щодо нотації: Mermaid не має вбудованого типу діаграми прецедентів, тому актора, прецеденти та межу системи побудовано засобами flowchart (актор – прямокутник зі заокругленими кутами, прецеденти – “stadium”-форма, межа системи – subgraph), із явним підписом стереотипів «include»/«extend» на зв’язках.

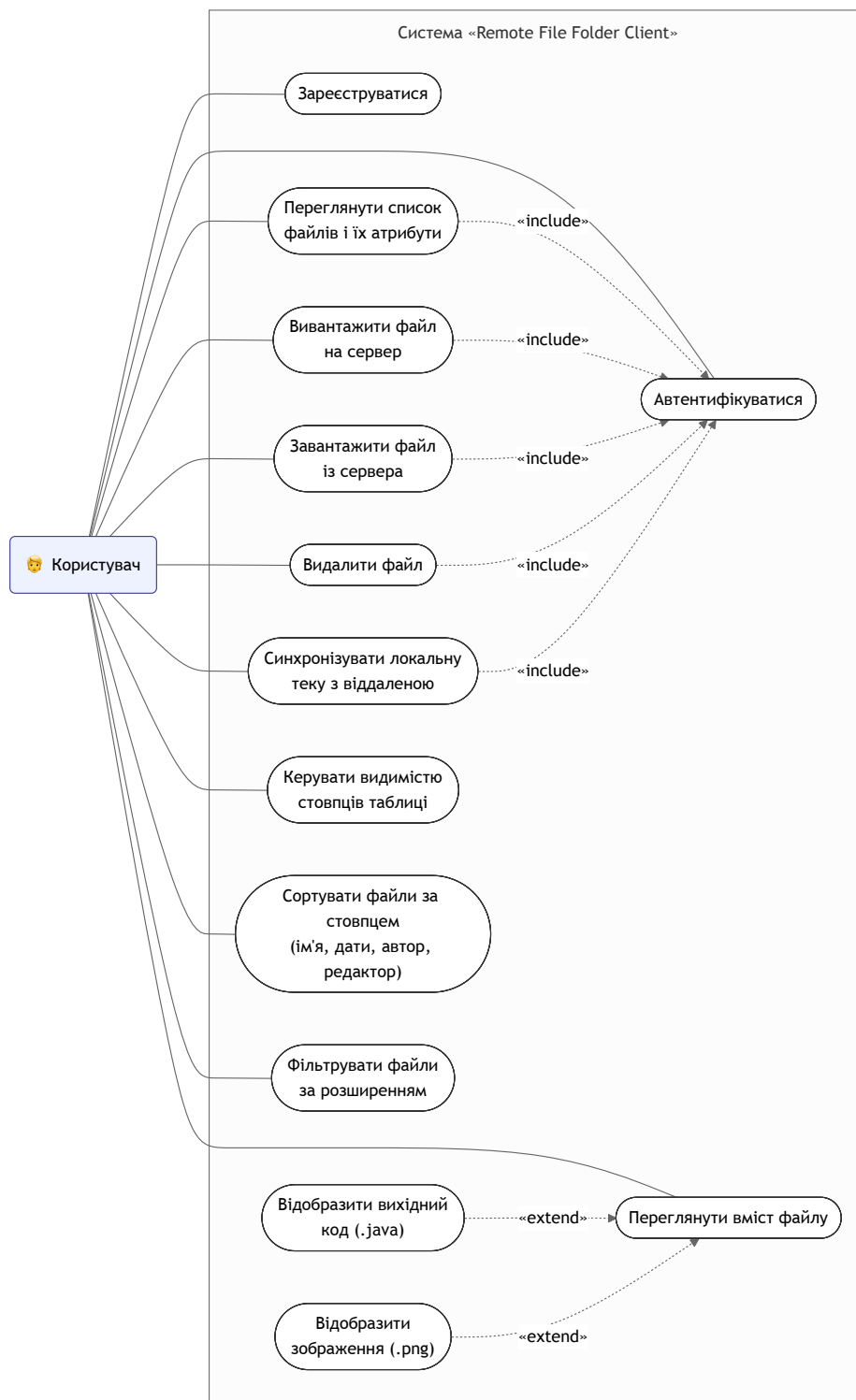


Рис. 1: Діаграма прецедентів системи Remote File Folder Client

2.2 Діаграма класів (загальна структура)

Рис. 2 відображає загальну структуру даних та основні взаємозв'язки між доменними класами системи, без деталізації кожного методу: сутності User та File (відповідно до backend/internal/domain/models.go – зауважте,

що User має username, а не email, а File не зберігає sha256), сутність “надгробок” Deletion, яка дозволяє sync-ендпойнту відрізнити “видалено” від “ще не завантажено”, сервісний шар (AuthService, FileService, SyncService), шар доступу до даних (FileRepo) та абстракцію сховища файлів FileStorage – відповідно до принципу “сховище за інтерфейсом”, закладеного в вимогах (розділ 5.2, 5.5). Наразі реалізовано лише LocalFileStorage; саме інтерфейс дозволив би згодом додати S3/R2-сумісне сховище без зміни коду викликів, але такої реалізації ще не існує, тож її свідомо не включено в діаграму. Так само автентифікація видає stateless JWT access/refresh токени напрямку, а не зберігає сутність Session – таблиці сесій чи списку відкликання на сервері немає.

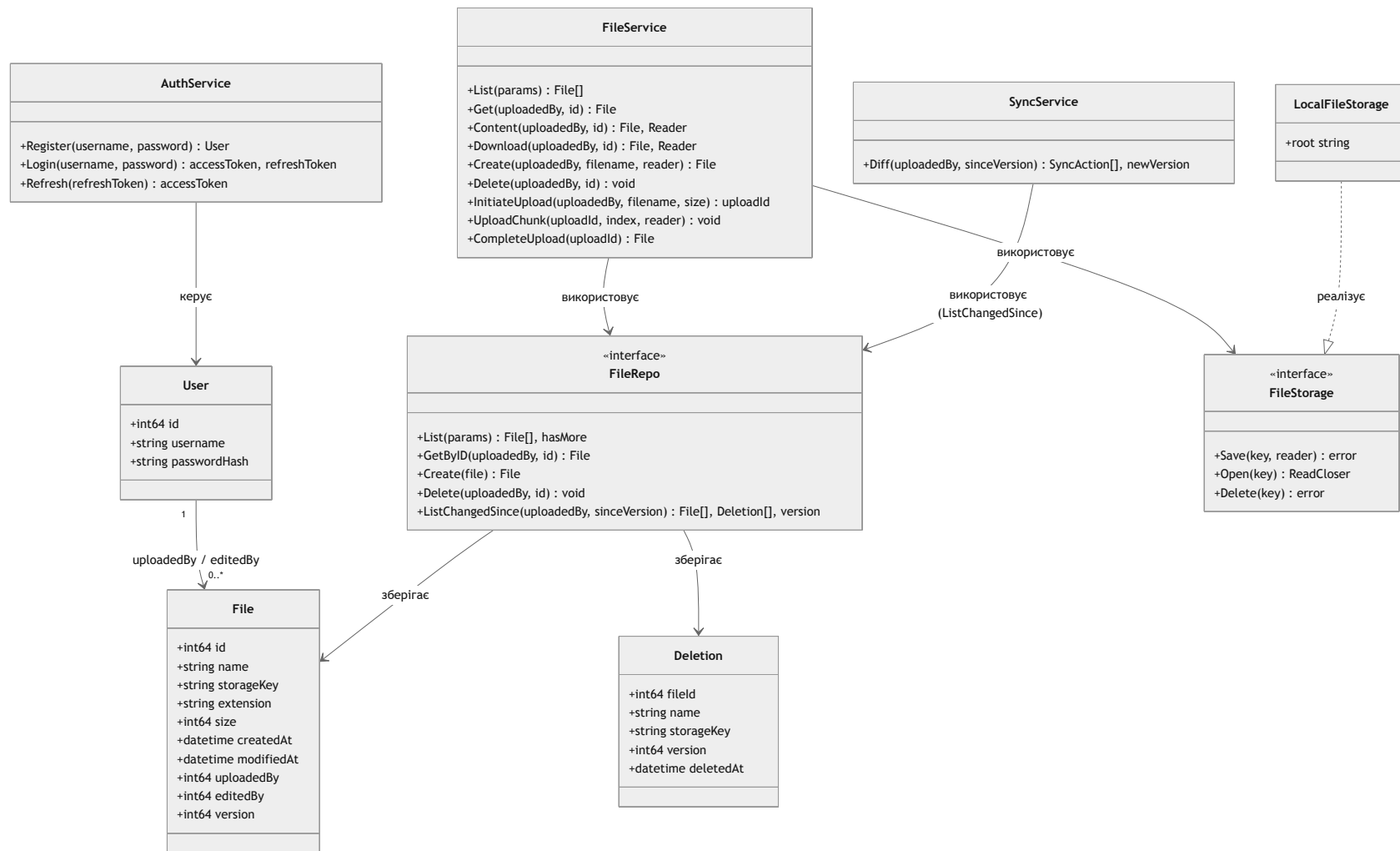


Рис. 2: Діаграма класів високого рівня (доменна модель)

2.3 Діаграма розгортання (загальна архітектура)

Рис. 3 відображає фізичну топологію системи: клієнтські пристрої (браузер, macOS), периферію Cloudflare (термінація TLS, публічний DNS) та домашній Ubuntu-сервер, що через Docker Compose підіймає чотири контейнери – `cloudflared` (вихідний тунель без відкриття портів, маршрутизує за шляхом), `api` (Go-бінарник), `web` (nginx, що роздає зібраний React SPA як статику) та `db` (PostgreSQL) – разом із постійним томом `pgdata` та `bind-mount` каталогом `../storage` для вмісту файлів. `cloudflared` спрямовує запити під `/api/*` до `api`, а все інше – до `web`, тож SPA й API поділяють один публічний хостнейм (див. `deploy/docker-compose.yml` та `deploy/cloudflared/confi`

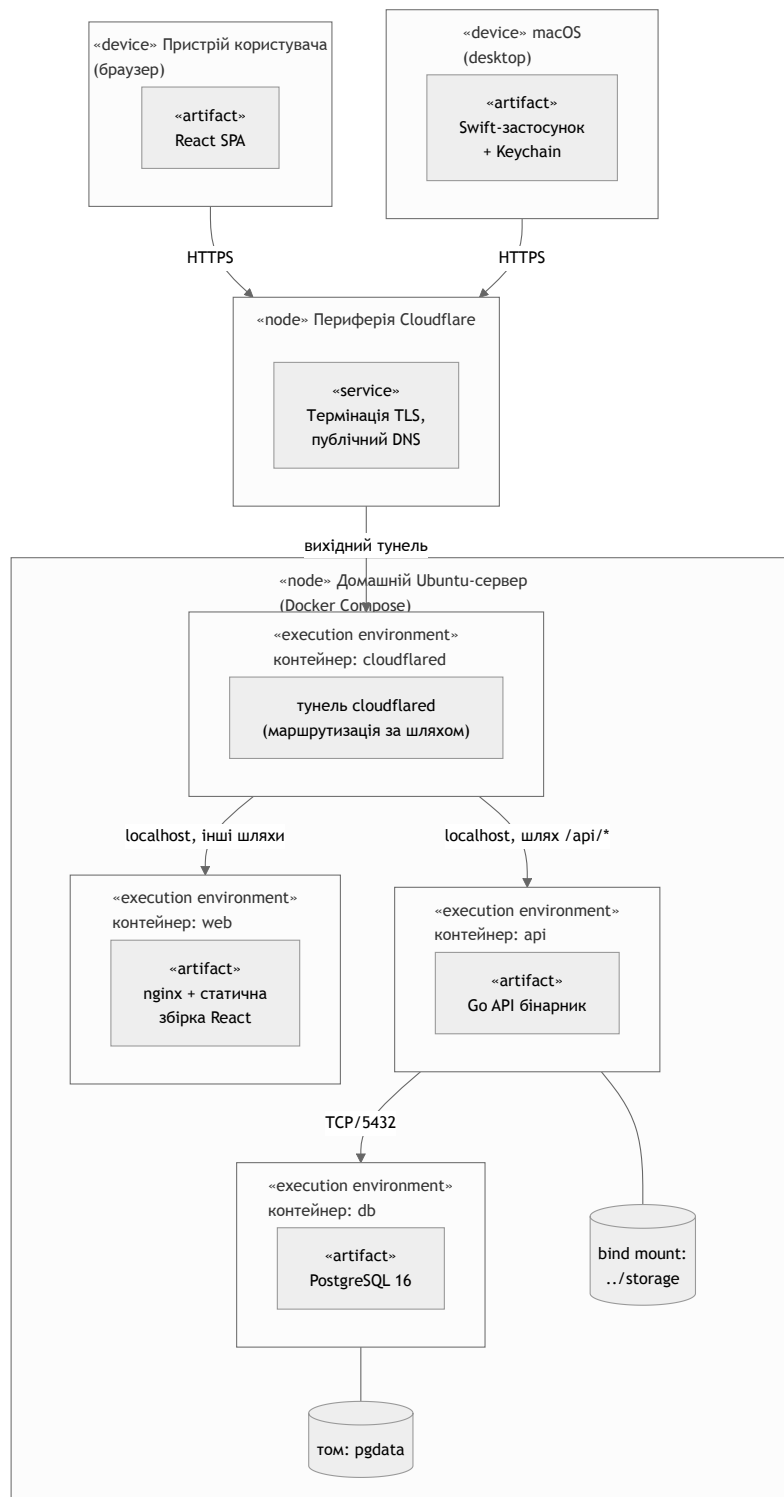


Рис. 3: Діаграма розгортання

3. Деталізоване проєктування

Деталізоване проєктування розкриває внутрішню структуру та поведінку окремих підсистем: класи, що беруть участь у реалізації конкретного прецедента (VOPC-діаграма), конкретний знімок станів об'єктів усередині

ні циклу узгодження (діаграма об'єктів), взаємодію об'єктів у часі – як у часовій (sequence), так і в структурній (communication) нотації – поведінку файлу залежно від стану синхронізації (діаграма станів), структуру програмних компонентів (діаграма компонентів), організацію коду за модулями (діаграма пакетів) та логіку бізнес-процесу синхронізації (діаграма активності).

3.1 VOPC-діаграма класів для прецедента “Синхронізувати локальну теку з віддаленою”

Згідно з варіантом завдання, як прецедент для VOPC-діаграми (*View Of Participating Classes*) обрано “Синхронізувати локальну теку з віддаленою” – найскладніший сценарій системи, тепер показаний із класами, які справді його реалізують у `desktop/Sources/FileSpaceDesktop/` та `backend/internal/handler,service/sync.go`. Класи розподілено за стандартними стереотипами аналізу:

- «boundary» – елементи, що взаємодіють з користувачем та ОС: `SyncPanelView` (обрати теку / start / stop), `DirectoryWatcher` (обгортка `FSEvents`, що генерує подію локальної зміни);
- «control» – керуюча логіка: `SyncViewModel` (цикл узгодження на клієнті), `FolderScanner` (обчислює локальний маніфест), `SyncHandler/SyncService` (сервер, односторонній diff);
- «entity» – дані предметної області: `SyncState` (збережений курсор і відомі позиції по файлах), `SyncedFileEntry`, `File`, `Deletion`.

На цій діаграмі свідомо немає `ConflictResolver` чи `ConflictDialogUI`: як пояснено в примітці розділу 1, сервер ніколи не повертає дію `conflict`, тож і клієнтського класу для її розв’язання не існує.

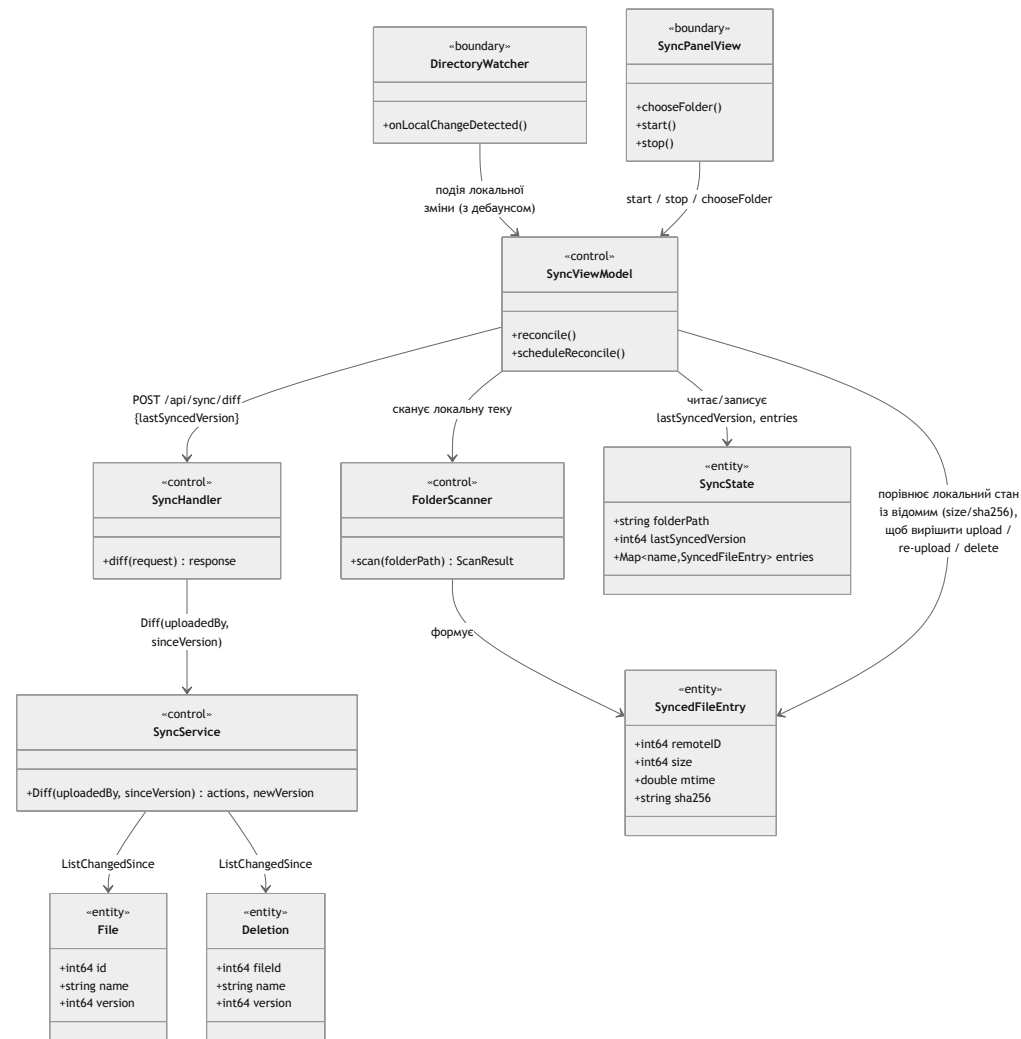


Рис. 4: VOPC-діаграма класів для прецедента “Синхронізувати локальну теку з віддаленою”

3.2 Діаграма об'єктів (знімок стану синхронізації)

На відміну від класових діаграм, що описують структуру типів, діаграма об'єктів (рис. 5) фіксує *конкретний знімок* екземплярів під час виконання – одну мить усередині `SyncViewModel.reconcile()`, коли локально змінений файл ось-ось буде повторно вивантажено. Збережений `state1 : SyncState` містить `lastSyncedVersion = 42` та відому позицію для `Notes.java` (`remoteID = 8241, size/sha256`, зафіксовані під час останньої синхронізації цього клієнта). `FolderScanner` формує свіжий `local : LocalFileEntry` для того самого файлу, чий `size/sha256` вже не збігаються з відомою позицією, тож `SyncViewModel` вирішує видалити віддалений файл 8241 і повторно вивантажити його як новий рядок (ендпойнта оновлення на місці немає). Як прямо зазначено в примітці на діаграмі, це *перегони (race)*, а не виявлений конфлікт: якби `remoteFile` теж змінився на сервері після версії 42, наступний же виклик `POST /api/sync/diff` незалежно повернув би для нього дію `download`, і жодна зі сторін так і не дізналася б, що інша теж змінилася.

Примітка щодо нотації: Mermaid не має вбудованого типу діаграми об'єктів, тому екземпляри побудовано засобами `flowchart`: заголовок кожного об'єкта підписано за конвенцією UML як ім'я_екземпляра : Клас, а нижче через розділювач перелічено конкретні значення атрибутів.

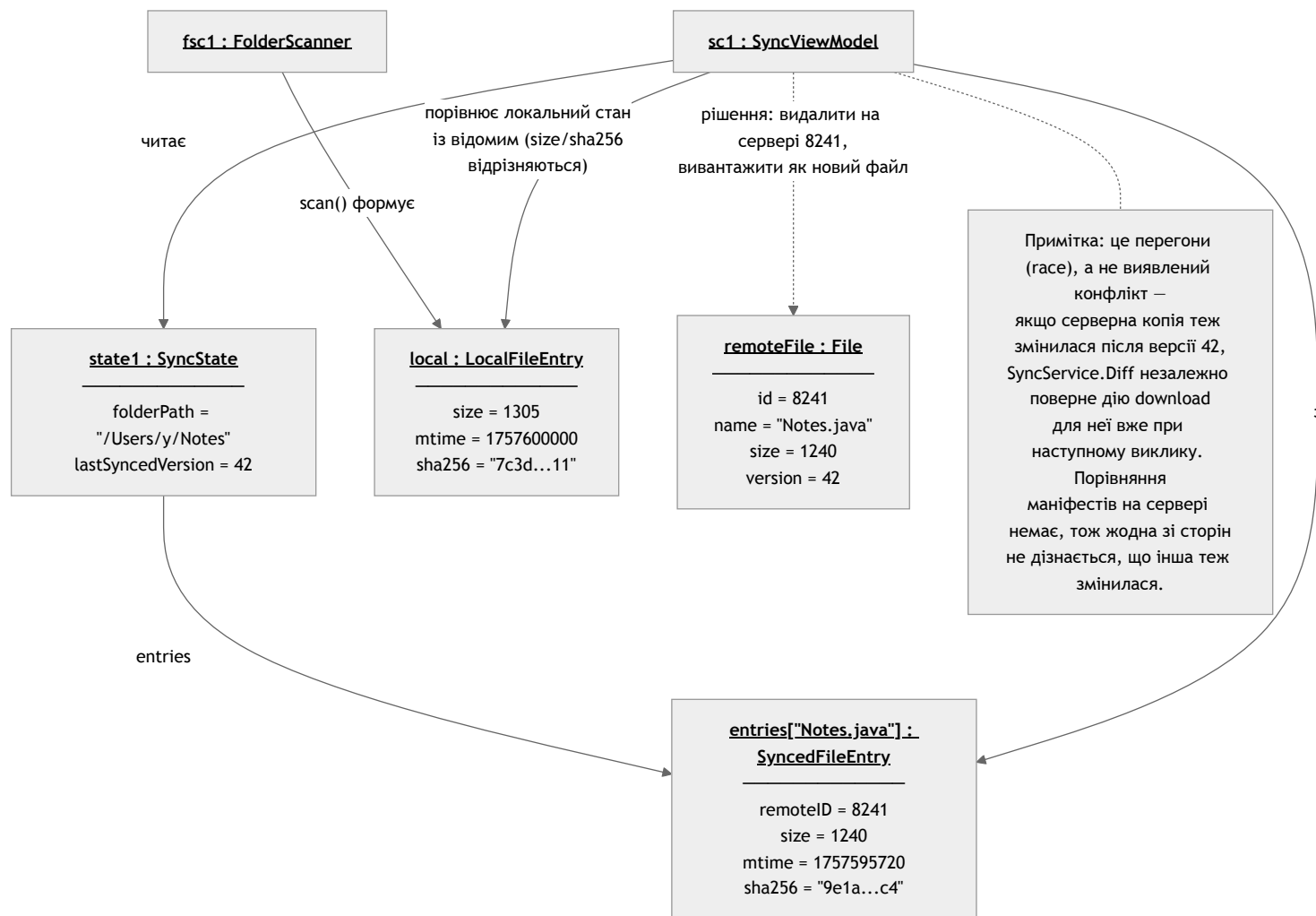


Рис. 5: Діаграма об'єктів: знімок стану синхронізації безпосередньо перед повторним вивантаженням локально зміненого файлу

3.3 Діаграми взаємодії (Sequence & Communication Diagrams)

Наведено три діаграми послідовності, що ілюструють ключові сценарії взаємодії клієнта, Go API-сервера та бази даних, а також комунікаційну діаграму як альтернативне представлення одного з цих сценаріїв.

Автентифікація

Рис. 6 показує послідовність перевірки облікових даних користувача (за username, не email) та видачі пари токенів (access/refresh) як двох незалежно підписаних, stateless JWT – жодного рядка сесії на сервері не створюється й не шукається.

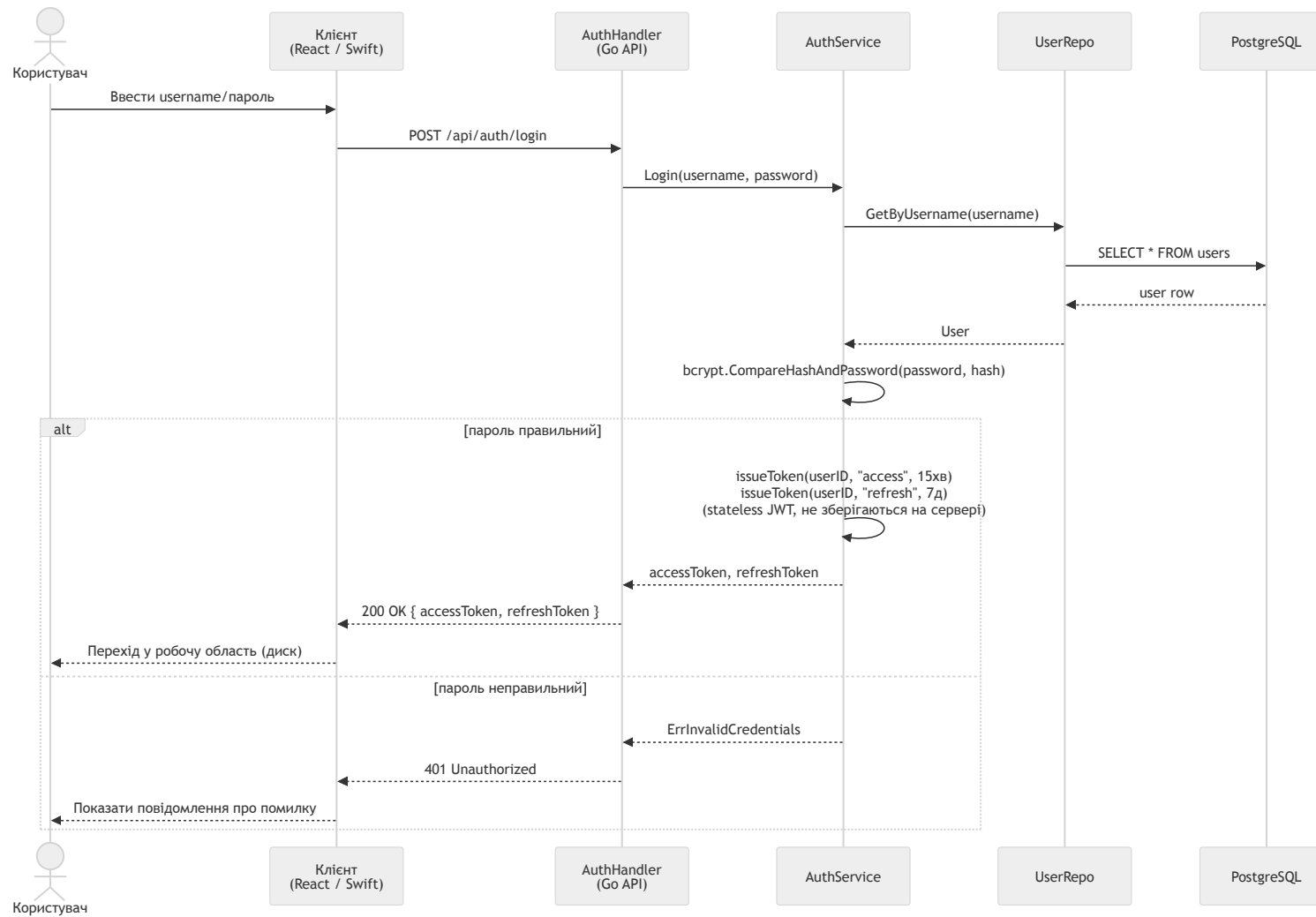


Рис. 6: Діаграма послідовності: автентифікація користувача

Вивантаження файлу

Рис. 7 ілюструє шлях запиту від клієнта через шар обробників (handler) до сервісного шару, який послідовно записує вміст файлу у сховище (FileStorage), а після підтвердження запису – метадані файлу у PostgreSQL через FileRepo, повторюючи спробу з автоматично згенерованим "name (n).ext", якщо назва конфліктує з уже наявною в цього користувача. Примітка на діаграмі вказує на chunked-варіант того самого шляху, який використовується для великих файлів.

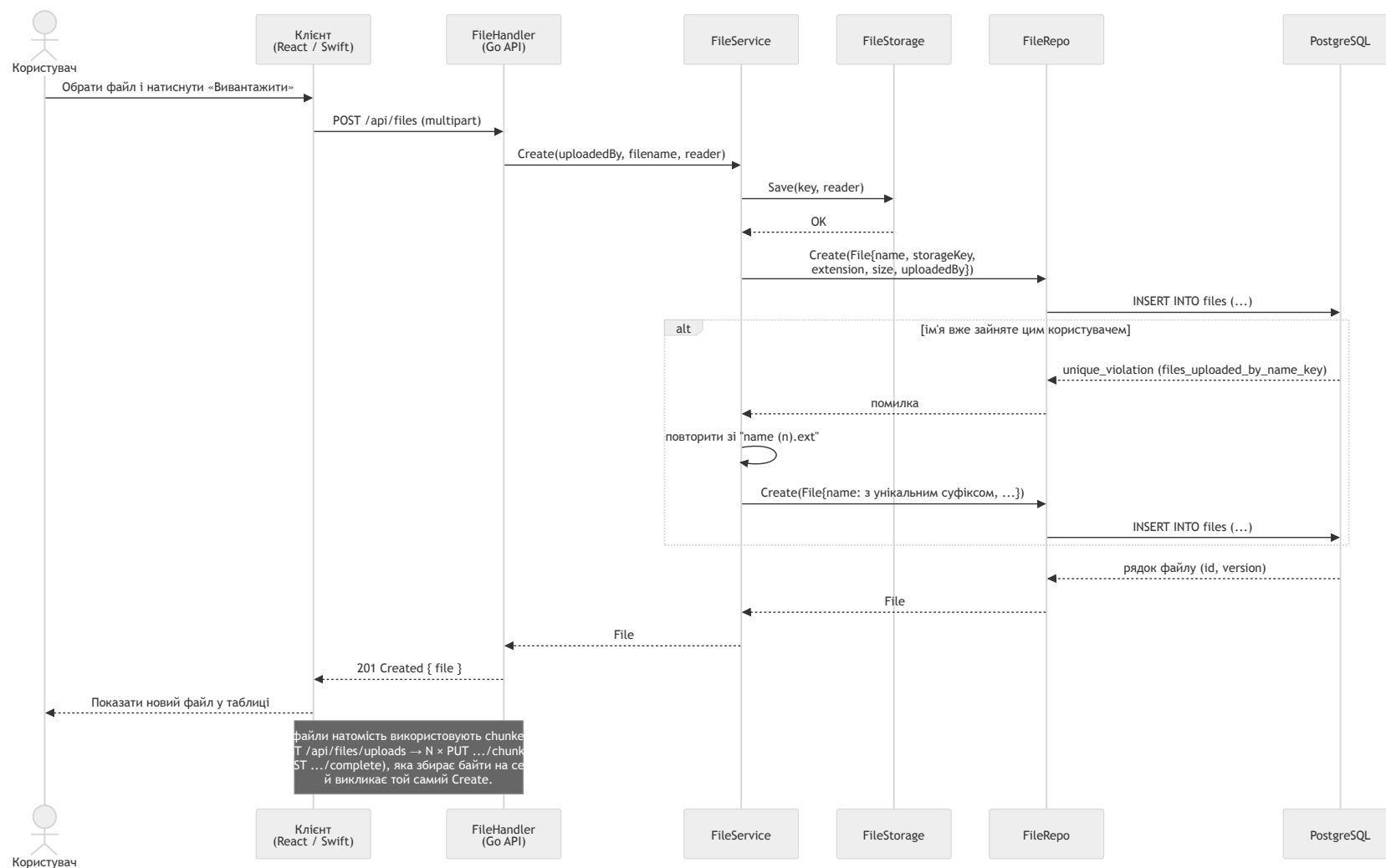


Рис. 7: Діаграма послідовності: вивантаження файлу на сервер

Синхронізація

Рис. 8 деталізує реальний обмін `POST /api/sync/diff`: клієнт надсилає лише `lastSyncedVersion` (маніфест на сервері не читається), сервер повертає дії `download/delete_local` разом із `newVersion`, клієнт їх застосовує, а потім – незалежно, без участі сервера в прийнятті рішення – сканує свою локальну теку й вивантажує, повторно вивантажує або видаляє файли на основі власного локально збереженого `SyncState`. Як зазначено в примітці на діаграмі, гілки для конфлікту немає: зміна одного файлу з обох боків між двома циклами розв’язується тим, яка половина цієї процедури виконується першою.

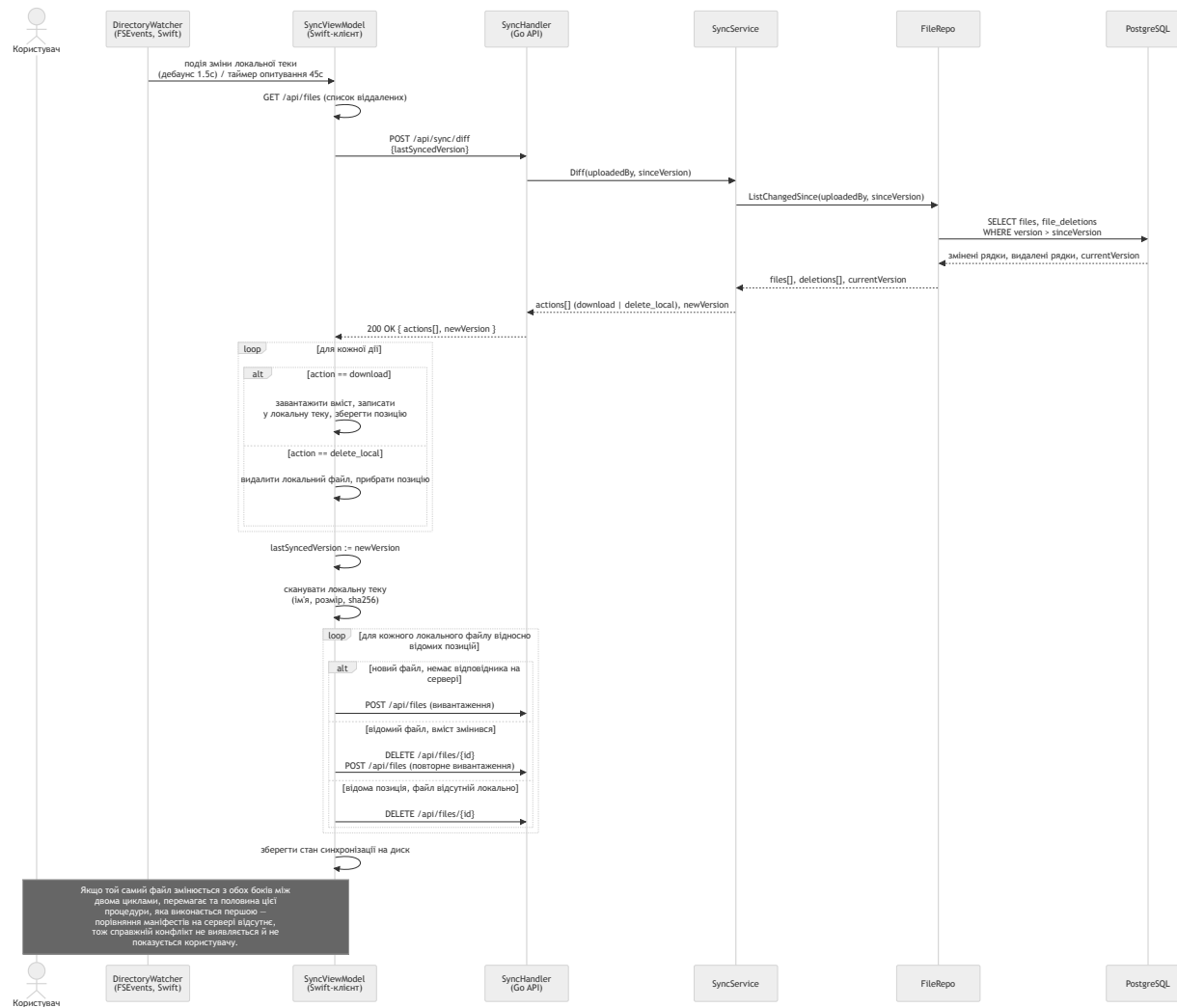


Рис. 8: Діаграма послідовності: синхронізація (серверний diff + узгодження на клієнті)

Той самий сценарій у нотації комунікаційної діаграми

Рис. 9 зображує **той самий** сценарій синхронізації, що й рис. 8, але в нотації комунікаційної (communication) діаграми: акцент зміщено з часової осі на статичні зв'язки між об'єктами, а порядок повідомлень передається нумерацією. Обидві діаграми узгоджені між собою – використовують ті самі об'єкти й повідомлення – і демонструють два стандартні способи показати одну взаємодію.

Примітка щодо нотації: Mermaid не має вбудованого типу комунікаційної діаграми, тому вона побудована засобами flowchart: об'єкти – вузли графа, а нумеровані підписи на зв'язках відповідають повідомленням.

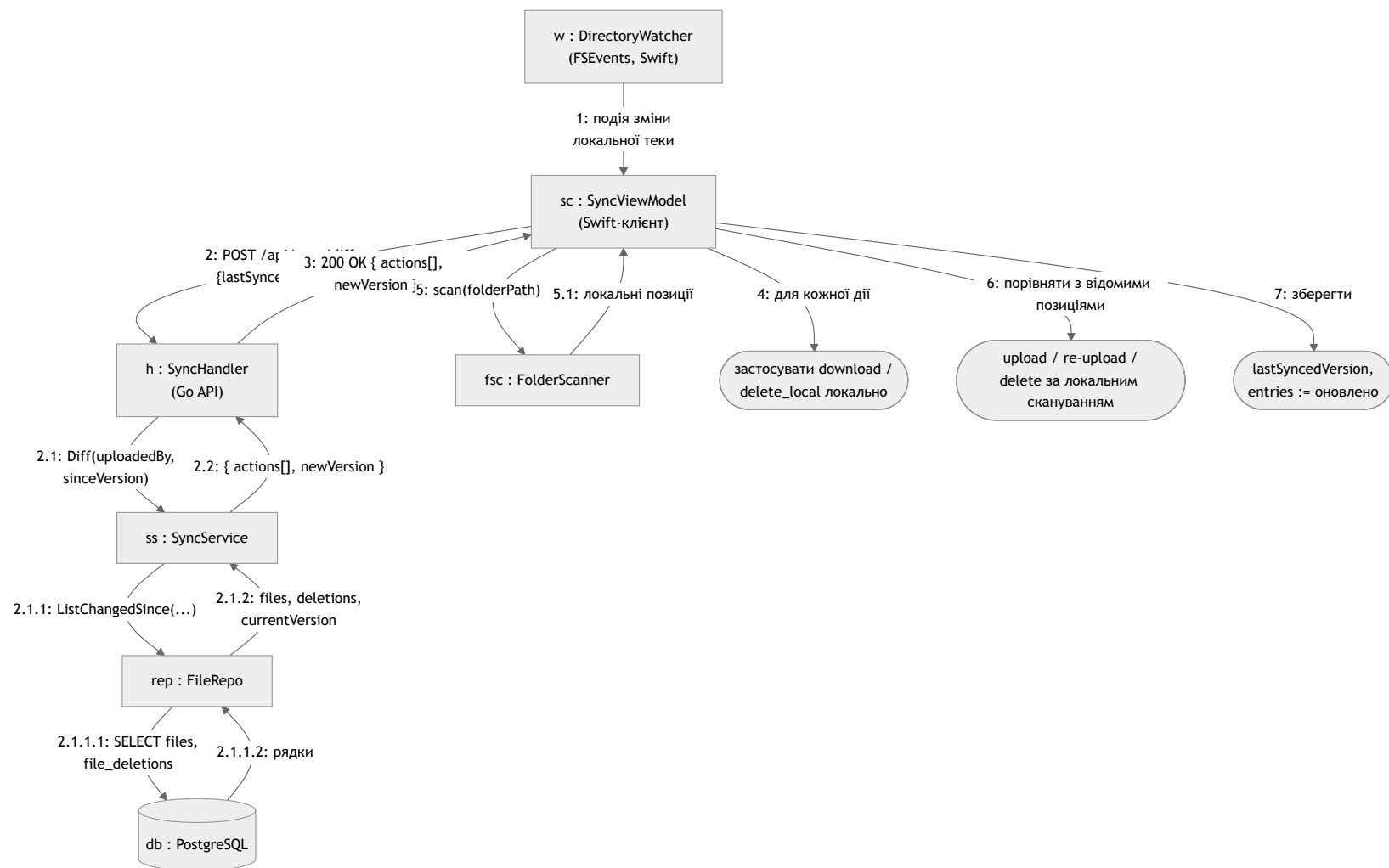


Рис. 9: Комунікаційна діаграма: синхронізація (серверний diff + узгодження на клієнті)

3.4 Діаграма станів

Рис. 10 описує поведінку файлу залежно від стану синхронізації з погляду клієнта: від локального/віддаленого створення, через передачу (Uploading/Download до Synced, можливих локальних чи віддалених змін (які повертають до повторного вивантаження або завантаження), і видалення з будь-якого боку (віддалене видалення приходить як дія `diff delete_local`; локальне видалення виявляється, коли відома позиція файлу зникає під час сканування теки). Як зазначено в примітці на діаграмі, стану Conflict свідомо немає: одночасна зміна з обох боків – це перегони, розв’язувані принципом last-writer-wins, а не змодельований перехід.

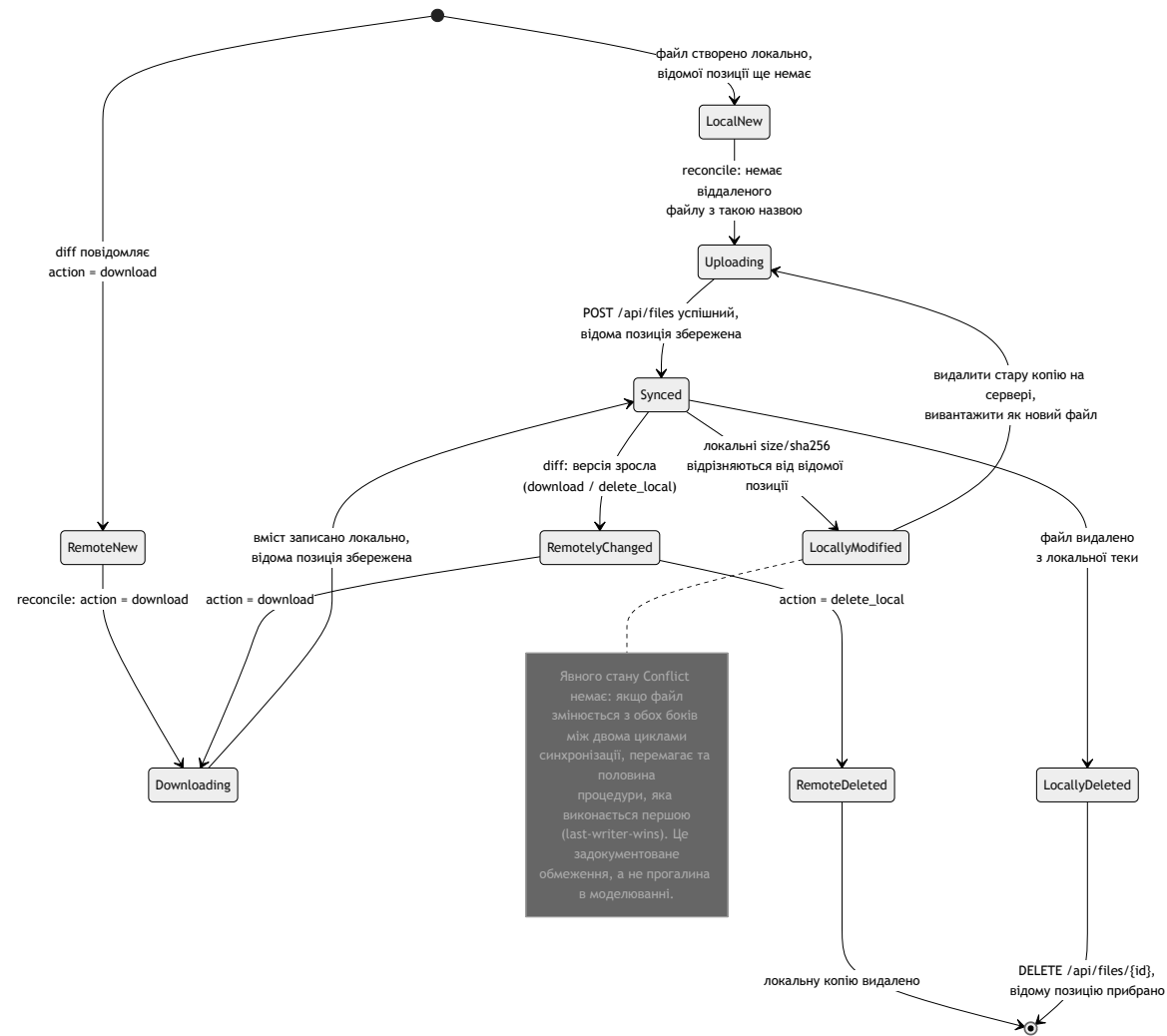


Рис. 10: Діаграма станів файлу в процесі синхронізації

3.5 Діаграма компонентів

Рис. 11 деталізує програмні компоненти системи та їх залежності: клієнтські застосунки, тунель `cloudflared` (що маршрутизує за шляхом до контейнера `api` або `web`), компонент `web` на `nginx`, який роздає зібраний React SPA, а всередині Go API-сервера – шари `Handler` → `Service` → `Repository`, а також незалежний від конкретної реалізації інтерфейс `FileStorage`, чия єдина наразі реалізація – локальне сховище на диску (розділ 5.2, 5.5 вимог).

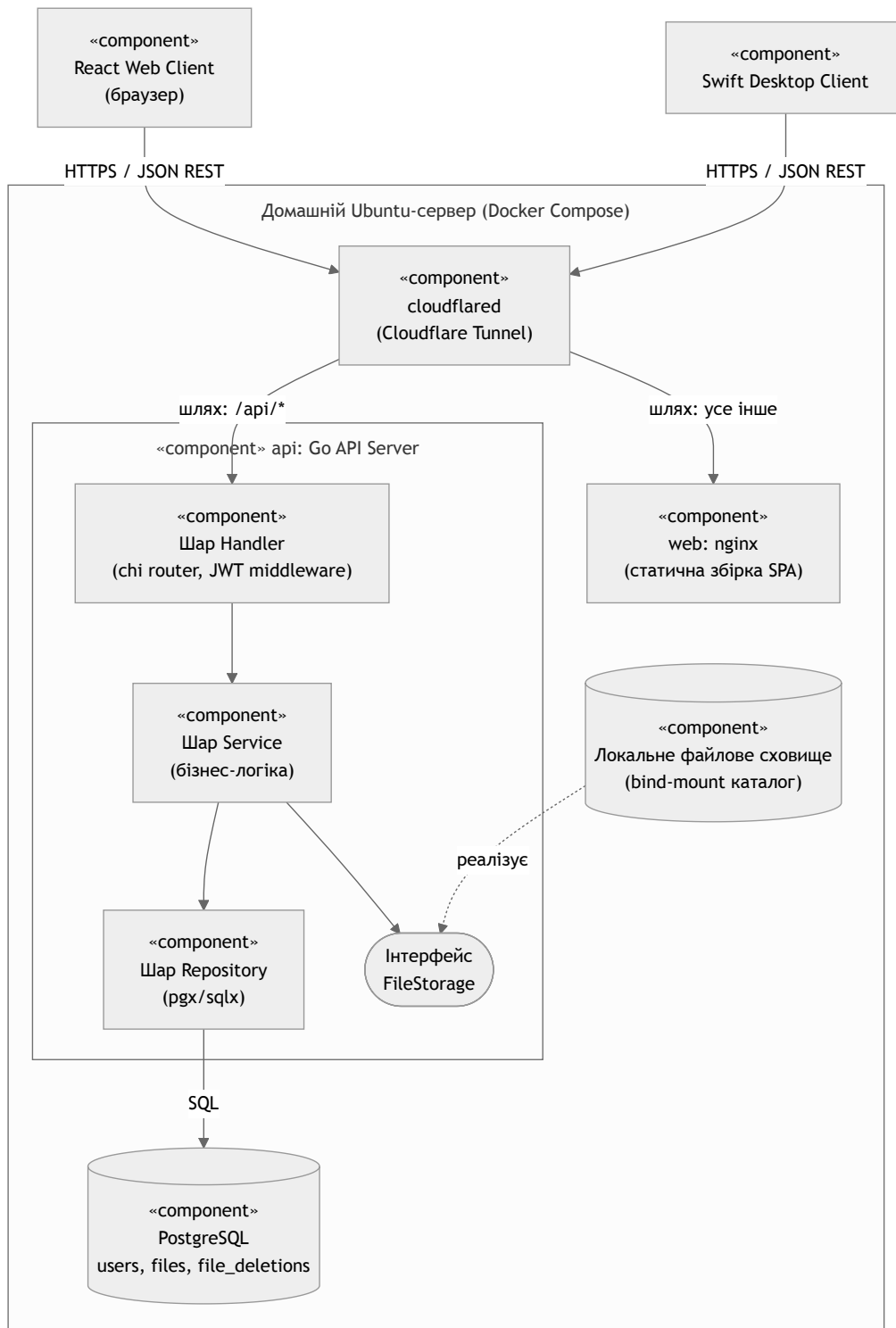


Рис. 11: Діаграма компонентів системи

3.6 Діаграма пакетів

Якщо діаграма компонентів (рис. 11) показує компоненти часу виконання та інтерфейси між ними, то діаграма пакетів (рис. 12) відображає *статичну* організацію коду за модулями відповідно до фактичної структури ре-

позиторію (backend, frontend, desktop, migrations): усередині Go-модуля – шарова структура пакетів handler → service → repo/storage, побудована на спільних допоміжних пакетах (domain, db, config, authctx), що відповідає конвенції “жодних прямих викликів файлової системи з handler” та принципу доступу до сховища лише через інтерфейс FileStorage. Пакет repo додатково залежить від схеми, визначеної в migrations. Пакет desktop окремо групує SwiftUI-екрани від рушія синхронізації (DirectoryWatcher, SyncViewModel та SyncState/FolderScanner з Manifest.swift) і спільних APIClient/KeychainStore, а пакет frontend тепер також залежить від TanStack Virtual для віртуалізації таблиці файлів поряд із TanStack Query та Table.

Примітка щодо нотації: Mermaid не має вбудованого типу діаграми пакетів, тому пакети зображено вкладеними subgraph-блоками (стереотип «package» у заголовку), а залежності – пунктирними стрілками з підписом дії.

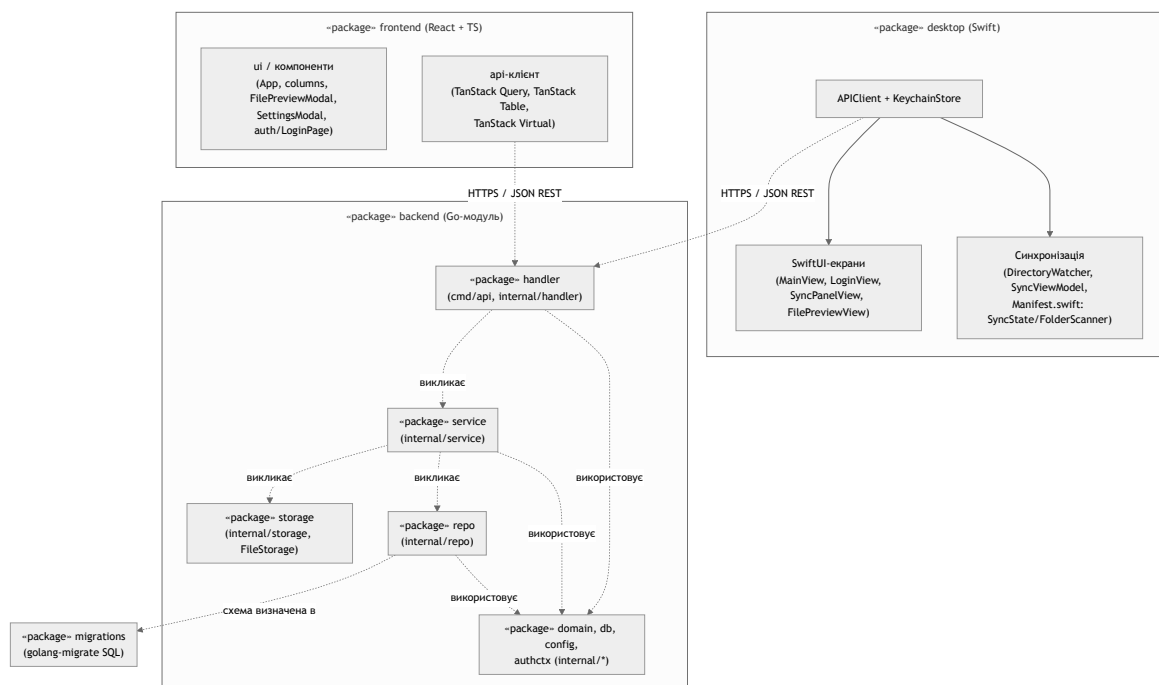


Рис. 12: Діаграма пакетів: організація коду за модулями

3.7 Діаграма активності

Рис. 13 деталізує бізнес-логіку одного циклу узгодження, що запускається або подією FSEvents, або опитуванням раз на 45 секунд: отримання списку віддалених файлів та запит серверного diff від lastSyncedVersion, застосування кожної дії download/delete_local і збереження нової версії, а потім сканування локальної теки та, незалежно від сервера, вивантаження нових файлів, видалення-і-повторне-вивантаження змінених файлів і видалення серверних копій файлів, видалених локально, перш ніж зберегти оновлений стан синхронізації на диск.

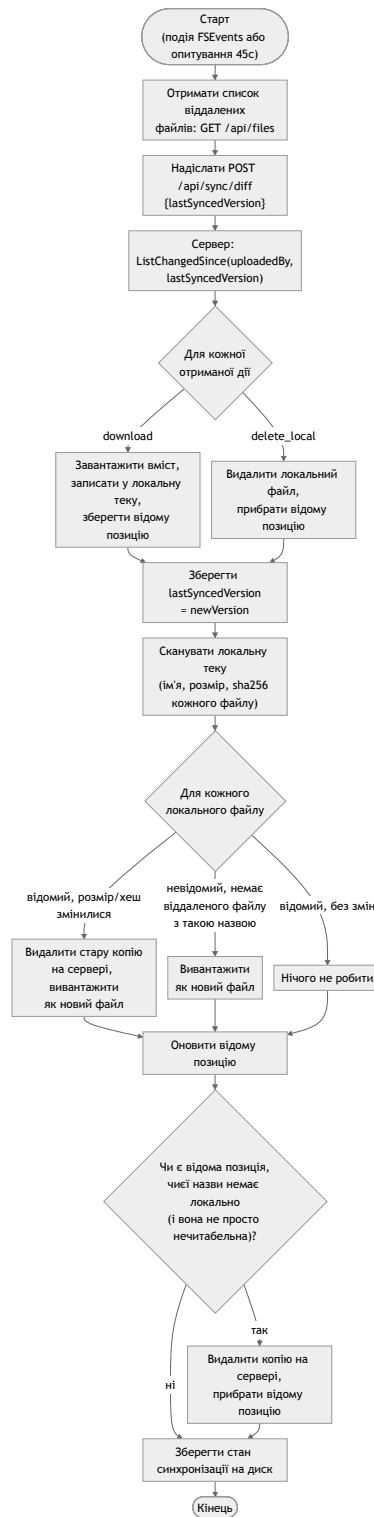


Рис. 13: Діаграма активності: цикл синхронізації

4. Висновки

Розроблений комплект діаграм послідовно розкриває систему Remote File Folder Client від бізнес-вимог до деталізованої структури і поведінки такою, якою вона реально реалізована, а не такою, як була первісно на-

креслена: діаграма прецедентів фіксує функціональні сценарії користувача, включно з реєстрацією; загальна діаграма класів і (тепер чотири-контейнерна) діаграма розгортання задають архітектурний каркас; VOPC-діаграма, діаграма об'єктів, діаграми послідовності та комунікаційна, станів, компонентів, пакетів та активності деталізують реалізацію найскладнішого сценарію – синхронізації файлів між локальною та віддаленою мережею. Цей сценарій виявився простішим за первісно закладений обсяг: сервер надає односторонній diff на основі версійного курсору (лише download/delete_lo а десктоп-клієнт самостійно узгоджує власні локальні зміни без виявлення конфліктів на основі маніфестів чи інтерфейсу їх розв'язання – це свідомий, задокументований компроміс, а не відсутня функція. Зокрема, діаграма об'єктів і комунікаційна діаграма доповнюють відповідно VOPC-діаграму та діаграми послідовності конкретним знімком стану (перегони при повторному вивантаженні, а не розв'язаний конфлікт) та альтернативною (структурною, а не часовою) нотацією того самого сценарію, а діаграма пакетів фіксує статичну модульну організацію коду, узгоджену з фактичною структурою репозиторію. Побудовані діаграми узгоджені між собою (спільні класи, сутності та дії) та слугують достовірною основою для подальшої розробки й тестування системи.